

CS 6888: Program Analysis and its Applications

Assignment 1: Automated Test Generation

Due: February 4th at 5 PM
Worth 10 points

1 Learning Objective

To consolidate your understanding of, and obtain hands-on experience with:

- how fuzzing tools work
- where fuzzing excels and where it struggles
- how to improve fuzzing performance
- what to do with the information produced

2 Individual Assignment

This is an individual assignment. You are not to share your setup, strategies, results, analysis, code, or any other aspect of the assignment with your classmates.

3 Environment

You will use the same image that you built for HW0. Launch it to execute the commands under practice below.

3.1 Core pattern on your own machine

If you are using `portal`, then you can ignore this.

Some distros have a binary processing core dumps - AFL will tell you if this is the case. If it does, you'll need to run the following command from the host to restore the standard behaviour. (This setting will only persist until the next reboot.)

```
echo core | sudo tee /proc/sys/kernel/core_pattern
```

4 Practice

For this assignment, our Program Under Test will be TinyCC. There are two versions of it compiled in the image - a normal version at `/root/tcc` and a version compiled with coverage support at `/root/tcc-cov`.

4.0.1 Demo run

In a shell, cd to `files/hw1/` and run `./run_afl.sh` - afl will start running and display useful information about its progress. The output will be saved to `files/hw1/afl_output/`. When you are ready to stop it, press `ctrl+c`.

Next, run `./run_aflcov.sh`. This will take the afl-fuzz output and compute coverage information from it. The starting point for viewing this information is `files/hw1/afl_output/cov/web/index.html`, which you can open on the host.

For the assignment, you may either modify these two scripts to change your parameters or you can create new ones.

4.0.2 Live mode

For exploration and early feedback, it may be useful to use live mode. The `--live` flag will start afl-cov in live mode so that you can watch coverage progress as afl fuzzes. When doing this, you can run afl-cov first, and then start afl-fuzz in a second shell. (After you stop afl, afl-cov will finish processing the data and generate the coverage reports before stopping as well.)

Note: we recommend not using live mode for your submission/`run_all.sh` in order to make mistakes less likely. If you do use it, make very sure that afl-cov will generate the final report within the time limit and not stay looping in the background.

4.0.3 Editing the Run String

One of the strengths of afl is its ability to test many different program configurations.

The two bash files above use the `tcc_run_string.txt` file to determine how they are going to run tcc.

The initial run string is:

```
/root/tcc/bin/tcc -I /root/tcc/include -o hello @@
```

This tells afl to test tcc by fuzzing the C programs that tcc will compile (as indicated by the `@@`).

You are welcomed and encouraged to change the contents of `tcc_run_string.txt` to help increase the coverage.

4.0.4 Increasing the Seed Pool

The seed files are located in `files/hw1/afl_input`. A simple `hello.c` program is provided as a starter, but you should explore adding additional seed files.

5 Tasks

5.1 Run AFL with different strategies.

After you are comfortable with afl and tcc, you are to:

1. Establish a baseline by running AFL on TCC for 10 minutes with the given seed example and the provided TCC parameters while observing the collected coverage information.
2. Implement strategies to get AFL to cover 60% of the lines of code in TCC. Basic strategies include running AFL for extended periods of time, enriching the seed pool to reach sections that are not covered, and exploring more TCC parameters besides the program to compile. Record the parameters you use for each run and what the resulting coverage is.

Some more advanced strategies are available here:

<https://afl-1.readthedocs.io/en/latest/tips.html#performance-tips>

An overall understanding of TCC would also be very helpful here. Its documentation is available at <https://bellard.org/tcc/tcc-doc.html>

3. Explore the functions not covered in the `i386-asm.c`, `tccpp.c`, and `tcctools.c` files, investigating why they are not covered and what it would take for them to be covered.

5.2 Write a report.

The report must be under 3 pages¹ in ieee-conference format² and include the following contents:

1. AFL
 - (a) Explain ³ the strategies used to increase the coverage generated by the AFL Fuzzer.
 - (b) Plot the number of lines covered by AFL with each new parameter/technique that you tried, starting with the given sample seed, and then showing when in time your strategies were introduced. In the plot, the x-axis contains

¹You shouldn't need a full 3 pages, but you won't be penalized until you exceed 3 pages.

²Templates are available at <https://www.ieee.org/conferences/publishing/templates.html> or <https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhncsfqn>.

³NB: When we ask for you to "explain", we expect a thoughtful, structured analysis that covers a) what you did, b) how you did it, and c) why it worked (or why you thought it would work).

descriptions of the parameters that were added/changed, the y-axis represents coverage, and annotations are used to mark changes in strategies.

- (c) Discuss what strategies were the most cost-effective and explain why that was the case.
- (d) Provide the LCOV code coverage report after implementing those strategies for AFL. (A screenshot of the tool output is sufficient, but it must be legible at default zoom. Use `figure*` if needed.)
- (e) If you found failures, explain how they manifested and what input led to them. If you did not, please theorize what types of failures may occur.
- (f) Name the functions not covered in the `i386-asm.c`, `tccpp.c`, and `tcctools.c` files, and explain why they are not covered and what it would take to cover them.

Note: all graphics must be legible at default zoom. (In multi-column latex layouts, you can add a star to the ‘figure’ environment to let images span both columns if you can’t change the graphic to be legible in a single column.

```
\begin{figure*}
  \centering
  \includegraphics[width=0.9\textwidth]{imagename}
  \caption{This is an image.}
\end{figure*}
```

)

6 Submission

6.1 Report

Submit your report through Canvas.

6.2 Code

Your repo will be cloned at the due date. Be sure that you have committed and pushed your code at that time.